

Roteiro de Atividades Práticas — Módulo 1

Programas, Processos e Threads em um PC

Roteiro de Atividades Práticas — Programas, Processos e Threads em um PC

(Passo a passo detalhado, código em C e Python, ferramentas gratuitas Windows/Linux)

Disciplina: Programação Paralela e Concorrente

Carga horária estimada: 100 min

Ferramentas: 100% gratuitas, com opção equivalente para Windows e para Linux em cada atividade

Como usar este documento

Cada atividade prática traz:

- Um **objetivo específico**;
- Um **passo a passo numerado** (o que fazer, na ordem exata);
- **Código em C** (quando o conceito depende de chamadas de sistema, como `fork()/pthread`) e **código em Python equivalente** (quando aplicável), para turmas que preferem uma linguagem mais acessível ou quando o Windows nativo não suporta a chamada de sistema em C;
- Comandos separados para [LINUX] **Linux** e [WINDOWS] **Windows**;
- O **resultado esperado**, para o aluno conferir se acertou.

Kit de ferramentas gratuitas (referência rápida)

Finalidade	[LINUX] Linux	[WINDOWS] Windows
Compilador C	GCC (build-essential)	MinGW-w64 ou GCC via WSL2
Observar processos/threads	<code>ps</code> , <code>htop</code> , <code>/proc</code>	Gerenciador de Tarefas, Process Explorer (Sysinternals)
Depurador	GDB	GDB (via MinGW) ou WinDbg (gratuito, SDK Windows)
Detectar race conditions	Valgrind (Helgrind/DRD), ThreadSanitizer	ThreadSanitizer (via WSL2) ou Application Verifier
Paralelismo simples	OpenMP (GCC)	OpenMP (MinGW ou Visual Studio Community)
Python	python3 + <code>threading/multiprocessing</code>	Python (python.org) + mesmas bibliotecas
IDE gratuita	VS Code, Geany	VS Code, Visual Studio Community

Recomendação prática: No caso de uso do Windows nos laboratórios, instale o **WSL2** (Windows Subsystem for Linux, gratuito e nativo da Microsoft) — isso permite rodar **todos** os comandos Linux deste roteiro sem precisar de dual-boot ou máquina virtual pesada. É o caminho de menor atrito. Para turmas que preferem não instalar nada em C, **todas as atividades de processos e threads têm uma versão equivalente em Python**, que roda igual em Linux e Windows.

Instalação do WSL2 (uma única vez, requer reinício):

```
ws1 --install
```

Após reiniciar, abrir o "Ubuntu" instalado e rodar:

```
sudo apt update && sudo apt install build-essential gdb htop valgrind python3 -y
```

Instalação do Python (Windows, se optar por não usar WSL2):

1. Acesse <https://www.python.org/downloads/> e baixe a versão mais recente.
2. Execute o instalador e **marque a caixa "Add python.exe to PATH"** antes de clicar em Install.
3. Abra o PowerShell e confirme com `python --version`.

Módulo 1 (Aula 1 — 100 min): Anatomia de um Sistema em Execução

Objetivos

- Diferenciar programa, processo e thread por meio de observação direta do SO.
- Aprender a usar ferramentas gratuitas de monitoramento (linha de comando, gráficas e em Python).

Ferramentas necessárias (gratuitas)

[LINUX] **Linux:** `ps`, `htop`, `pstree`, `/proc` (nativos do sistema) + Python 3 (já instalado na maioria das distribuições).

[WINDOWS] **Windows:** Gerenciador de Tarefas, PowerShell (nativos) + **Process Explorer** (Sysinternals, gratuito, portátil) + Python (instalar em python.org, marcando "Add python.exe to PATH").

Biblioteca Python usada neste módulo: `psutil` — biblioteca gratuita e multiplataforma (funciona igual em Linux e Windows) para consultar processos, threads, memória e CPU. Instalação (uma única vez):

```
pip install psutil
```

No Windows, use `pip install psutil` no PowerShell (com o Python já no PATH).

*Por que usar Python aqui? As atividades deste módulo são de observação do sistema operacional. Os comandos nativos (`ps`, Gerenciador de Tarefas) já resolvem isso, mas escrever essas mesmas consultas em Python com `psutil` tem duas vantagens didáticas: (1) o código é **idêntico em Linux e Windows**, e (2) os alunos passam a enxergar que tudo que o `ps`/Gerenciador de Tarefas mostra na tela é, na verdade, informação que também pode ser obtida programaticamente — o que prepara o terreno para os módulos seguintes, onde vocês mesmos criarão processos e threads via código.*

Atividade 1.1 — Programa vs. Processo (20 min)

Objetivo da atividade: comprovar que abrir várias vezes o mesmo programa cria vários processos independentes.

Passo a passo:

1. Escolha um aplicativo simples (Bloco de Notas no Windows, ou `gedit/kate` no Linux).
2. Abra esse aplicativo **três vezes seguidas**, criando três janelas independentes na tela.
3. Abra um terminal (ou o Gerenciador de Tarefas) e liste os processos filtrando pelo nome do aplicativo.
4. Anote o PID de cada uma das três janelas em uma tabela (Janela 1, Janela 2, Janela 3 → PID).
5. Feche uma das janelas e repita a listagem — confirme que o PID correspondente desapareceu da lista.

6. Repita a consulta agora usando o script Python fornecido, e confirme se os PIDs batem com os observados nos passos anteriores.

Ferramentas de linha de comando:

[LINUX] Linux:

```
ps aux | grep gedit
# ou, para uma visão mais amigável:
pgrep -a gedit
```

[WINDOWS] Windows:

- Abrir o **Gerenciador de Tarefas** (Ctrl+Shift+Esc) → aba "Detalhes" → localizar o processo pelo nome (ex.: `notepad.exe`) → coluna PID.
- Alternativa mais poderosa (gratuita): **Process Explorer**, da Microsoft Sysinternals — <https://learn.microsoft.com/sysinternals/downloads/process-explorer> (não requer instalação, é um `.exe` portátil).

Script em Python (idêntico em Linux e Windows):

```
import psutil

# Troque pelo nome do processo do seu aplicativo:
# Linux: "gedit" | Windows: "notepad.exe"
nome_alvo = "gedit"

print(f"Processos encontrados para '{nome_alvo}':")
for processo in psutil.process_iter(["pid", "name", "ppid"]):
    if nome_alvo.lower() in processo.info["name"].lower():
        print(f"PID={processo.info['pid']} PPID={processo.info['ppid']}")
Nome={processo.info['name']}")
```

[LINUX] Linux: `python3 listar_processos.py`

[WINDOWS] Windows: `python listar_processos.py`

Resultado esperado: três PIDs diferentes, todos referenciando o mesmo executável/programa em disco — tanto na ferramenta nativa quanto no script Python. Ao fechar uma janela, o PID correspondente some das duas listagens.

Atividade 1.2 — Explorando threads de um processo (25 min)

Objetivo da atividade: verificar que um único processo pode conter múltiplas threads internas.

Passo a passo:

1. Descubra o PID do seu terminal atual (ou do navegador, se preferir um processo com mais threads).
2. Consulte quantas threads esse processo possui, usando a coluna/campo apropriado da ferramenta nativa.
3. Se a ferramenta permitir, liste as threads individualmente (cada uma com seu próprio identificador).
4. Repita a consulta em outro processo do sistema (ex.: o navegador) e compare o número de threads entre os dois processos.
5. Repita as duas consultas usando o script Python fornecido, e confirme se os números batem.
6. Registre os números encontrados em uma pequena tabela (Processo → N° de threads).

Ferramentas de linha de comando:

[LINUX] Linux:

```
# Passo 1: descobrir o PID do terminal atual
echo $$

# Passo 2: ver número de threads (coluna NLWP = Number of LightWeight Processes)
ps -o pid,ppid,nlwp,cmd -p <PID>

# Passo 3: ver o detalhe via /proc (cada subpasta numerada = 1 thread)
cat /proc/<PID>/status | grep -i threads
ls /proc/<PID>/task/
```

[WINDOWS] Windows:

- **Process Explorer** (Sysinternals, gratuito): selecionar o processo → clique direito → **Properties** → aba **Threads**, mostra a lista completa com IDs individuais.
- Alternativa via linha de comando (PowerShell, nativo, gratuito):

```
Get-Process -Id <PID> | Select-Object -ExpandProperty Threads
```

Script em Python (idêntico em Linux e Windows):

```
import psutil
import os

pid = os.getpid() # PID do próprio script Python em execução
# Para consultar outro processo, troque por: pid = <PID_desejado>

processo = psutil.Process(pid)
print(f"Processo PID={pid} - Nome: {processo.name()}")
print(f"Número de threads: {processo.num_threads()}")

print("\nDetalhe de cada thread (id, tempo de usuário, tempo de sistema):")
for t in processo.threads():
    print(f" Thread ID={t.id} tempo_usuario={t.user_time:.4f}s
tempo_sistema={t.system_time:.4f}s")
```

[LINUX] Linux: `python3 threads_do_processo.py`

[WINDOWS] Windows: `python threads_do_processo.py`

Resultado esperado: o processo terá tipicamente entre 1 e dezenas de threads, dependendo do software; navegadores costumam ter muito mais threads que um terminal simples. O script Python deve mostrar o mesmo número de threads observado na ferramenta nativa.

Atividade 1.3 — Árvore de processos (20 min)

Objetivo da atividade: visualizar a hierarquia pai-filho de processos no sistema e identificar o processo raiz.

Passo a passo:

1. Liste a árvore completa de processos do sistema usando a ferramenta nativa.
2. Localize o processo raiz (o único sem processo pai visível, ou cujo pai é ele mesmo).
3. Escolha um processo qualquer em execução (ex.: o terminal) e trace, subindo na árvore, sua cadeia de processos ancestrais até chegar à raiz.
4. Repita essa mesma reconstrução usando o script Python fornecido, que sobe automaticamente a cadeia de pais.
5. Desenhe essa cadeia (ex.: `systemd → bash → vim`) no relatório.

Ferramentas de linha de comando:

[LINUX] Linux:

```
pstree -p
# ou, focando em um processo específico:
pstree -p <PID>
```

[WINDOWS] Windows:

- **Process Explorer** exibe nativamente a árvore de processos (indentação mostra hierarquia pai-filho) — não requer instalação.
- Alternativa via PowerShell (nativo):

```
Get-CimInstance Win32_Process | Select-Object ProcessId, ParentProcessId, Name | Format-Table
```

Script em Python (idêntico em Linux e Windows):

```
import psutil
import os

def imprimir_cadeia_ancestrais(pid):
    cadeia = []
    try:
        processo = psutil.Process(pid)
        while processo is not None:
            cadeia.append(f"{processo.name()} (PID={processo.pid})")
            processo = processo.parent()
    except psutil.NoSuchProcess:
        pass

    # Imprime da raiz até o processo original
    print(" -> ".join(reversed(cadeia)))

pid_atual = os.getpid()
imprimir_cadeia_ancestrais(pid_atual)
```

[LINUX] Linux: `python3 arvore_processos.py`

[WINDOWS] Windows: `python arvore_processos.py`

Resultado esperado: identificar o processo raiz (Linux: `init/systemd`, PID 1; Windows: normalmente `explorer.exe` ou um processo de sistema) como ancestral comum, tanto na ferramenta nativa quanto na saída do script Python.

Atividade 1.4 — Monitoramento em tempo real com carga de CPU (25 min)

Objetivo da atividade: observar visualmente o efeito de um processo consumindo CPU intensivamente.

Passo a passo:

1. Abra um monitor de sistema em tempo real em uma janela.
2. Em outra janela/terminal, inicie um comando (ou script Python) que gere carga contínua de CPU.
3. Volte ao monitor e observe: o novo processo aparecendo na lista, o percentual de CPU subindo (idealmente perto de 100% em um núcleo), e o PID do processo.
4. Deixe rodando por 30 segundos, observando a variação.
5. Encerre manualmente o processo e confirme, no monitor, que o consumo de CPU volta a cair.
6. Repita o experimento usando o script de monitoramento em Python fornecido, rodando em paralelo com o gerador de carga.

Ferramentas de linha de comando (gerar carga):

[LINUX] Linux:

```
# Terminal 1 – monitor em tempo real (gratuito, geralmente já instalado ou via apt)
htop

# Terminal 2 – gerar carga de CPU
yes > /dev/null &
echo "PID gerado: $!"

# Encerrar ao final:
kill $!
```

[WINDOWS] Windows:

- Monitor: **Gerenciador de Tarefas** (aba Desempenho, gratuito e nativo) ou **Process Explorer**.
- Gerar carga de CPU (PowerShell, sem instalar nada):

```
Start-Job { while ($true) { } }
# Ver o job:
Get-Job
# Encerrar:
Stop-Job -Id <ID>; Remove-Job -Id <ID>
```

Script em Python para gerar carga de CPU (idêntico em Linux e Windows):

```
# gerador_carga.py – roda até ser interrompido com Ctrl+C
import os

print(f"Gerando carga de CPU. PID deste processo: {os.getpid()}")
print("Pressione Ctrl+C para interromper.")

x = 0
while True:
    x += 1
```

Script em Python para monitorar o processo em tempo real (rode em outro terminal, passando o PID anotado acima):

```
# monitor_cpu.py
import psutil
import time
import sys
```

```
pid = int(sys.argv[1]) # informar o PID como argumento na linha de comando
processo = psutil.Process(pid)

print(f"Monitorando PID={pid} ({processo.name()})")
for _ in range(10):
    uso_cpu = processo.cpu_percent(interval=1)
    print(f"Uso de CPU: {uso_cpu:.1f}%")
```

[LINUX] **Linux:**

```
python3 gerador_carga.py &
python3 monitor_cpu.py <PID_impresso>
```

[WINDOWS] **Windows (dois terminais PowerShell):**

```
python gerador_carga.py
# Em outro terminal:
python monitor_cpu.py <PID_impresso>
```

Resultado esperado: uso de CPU sobe para próximo de 100% em um núcleo enquanto o processo roda (visível tanto no monitor nativo quanto no script Python); volta a cair para próximo de 0% assim que ele é encerrado.

Questões para o relatório (Módulo 1)

1. Qual a diferença entre PID e PPID? Ilustre com um exemplo observado na Atividade 1.3.
2. Ao abrir três instâncias do mesmo programa, quantos PIDs você observou? Eles compartilham memória entre si?
3. Quantas threads o processo analisado na Atividade 1.2 possuía? Isso foi surpreendente?
4. Os resultados obtidos pelo script Python (`psutil`) bateram com os resultados da ferramenta nativa (`ps`/Gerenciador de Tarefas) em todas as atividades? Alguma diferença encontrada?
5. Qual ferramenta você usou (linha de comando nativa, gráfica, ou Python) e por quê? Descreva um obstáculo prático encontrado na instalação/uso (ex.: instalação do `psutil`).
6. Com base na experiência prática, escreva com suas próprias palavras a diferença entre processo e thread.